

TOPIC 07

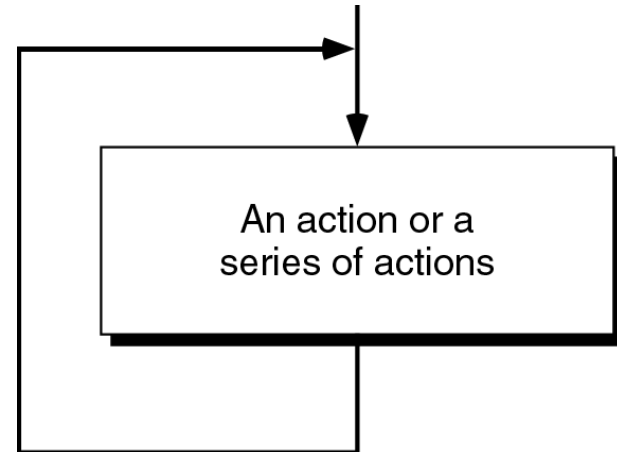
BRANCHING AND LOOP (PART 2)

Fundamentals of Computing
(FSPK0022)

Loop/ Repetition

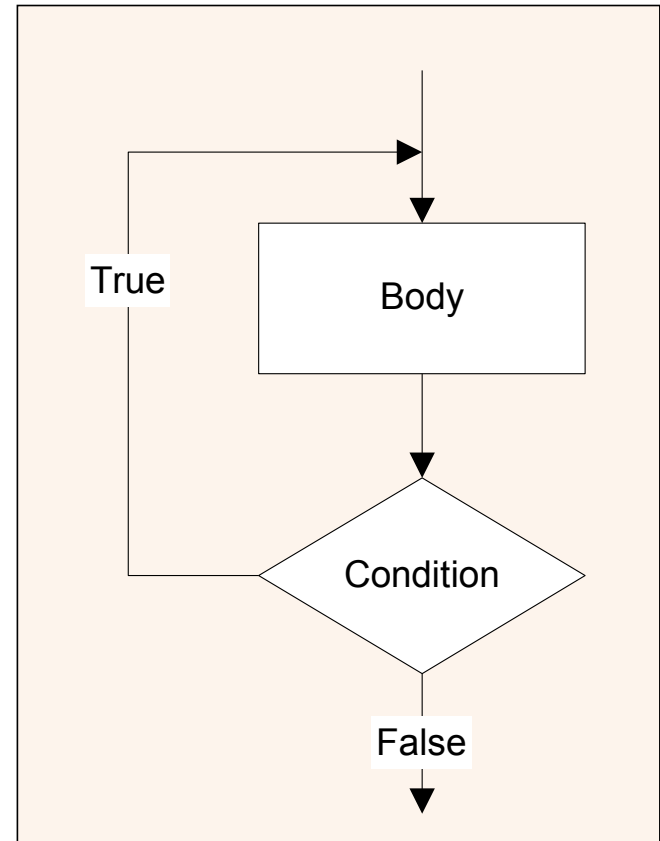
Loop/ Repetition

- The main idea of a loop is to **repeat an action** or a **series of actions**.
- But, when to stop looping?
- In the following flowchart, the action is executed over and over again. Its never stop.
 - This is called an **infinite loops**.
- Solution: put a **condition** to tell the loop either to continue looping or stop.



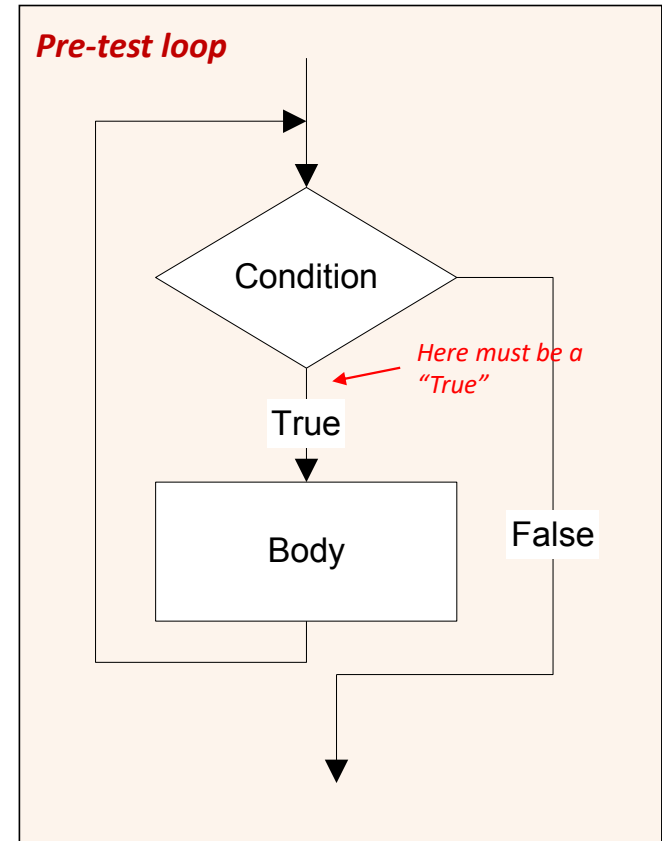
Loop

- A loop has two parts: **body** and **condition**.
- **Body**: a statement or a block of statements that will be repeated.
- **Condition**: is used to control the iteration, either to continue or stop iterating.



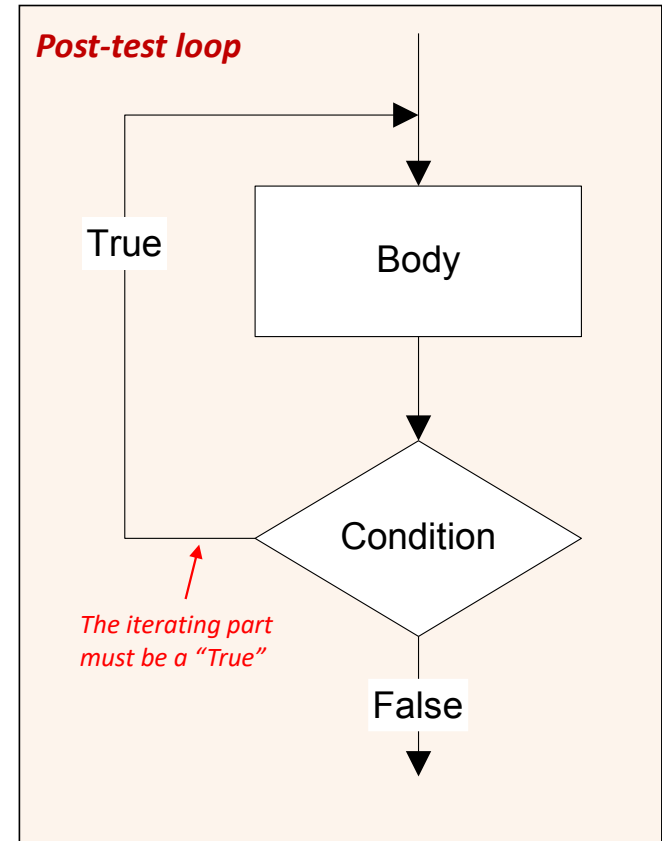
Types of Loop

- Two forms of loop:
 - Pre-test loop
 - Post-test loop
- **Pre-test loop**
 - The condition is tested first, before we start executing the body.
 - The body is executed, if the condition is true.
 - After executing the body, the loop repeats.



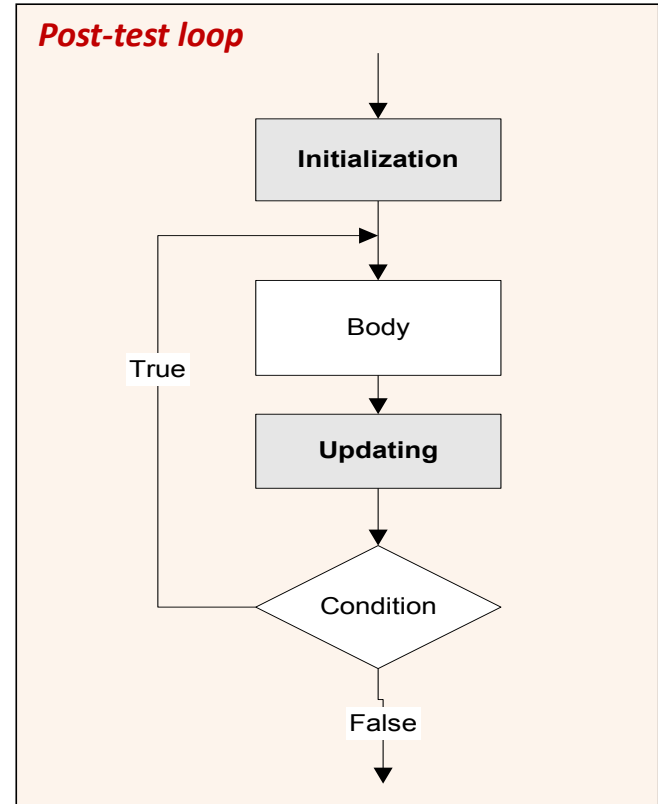
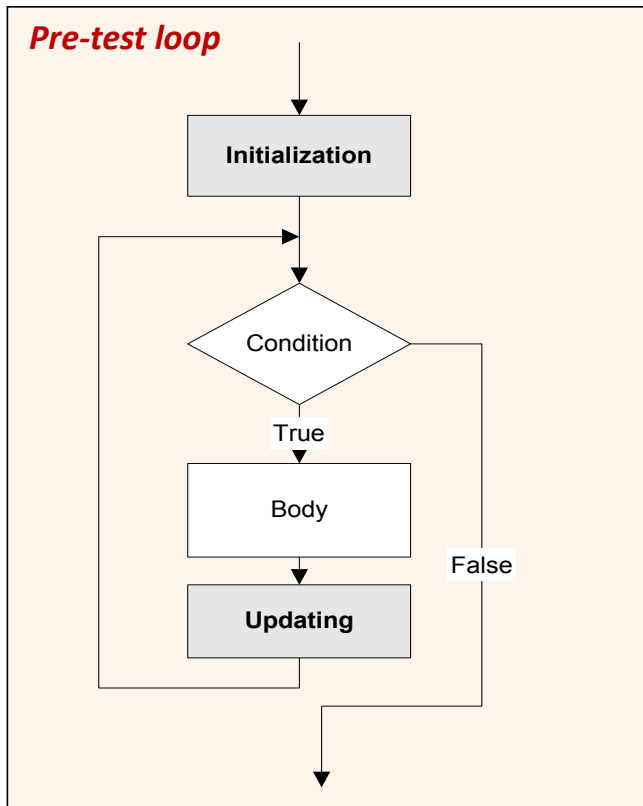
Types of Loop

- **Post-test loop**
 - The condition is tested later, after executing the body.
 - If the condition is true, the loop repeats, otherwise it terminates.
 - The body is always executed at least once.



Parts of a Loop

- Beside the body and condition, a loop may have two other parts: **initialization** and **updating**.



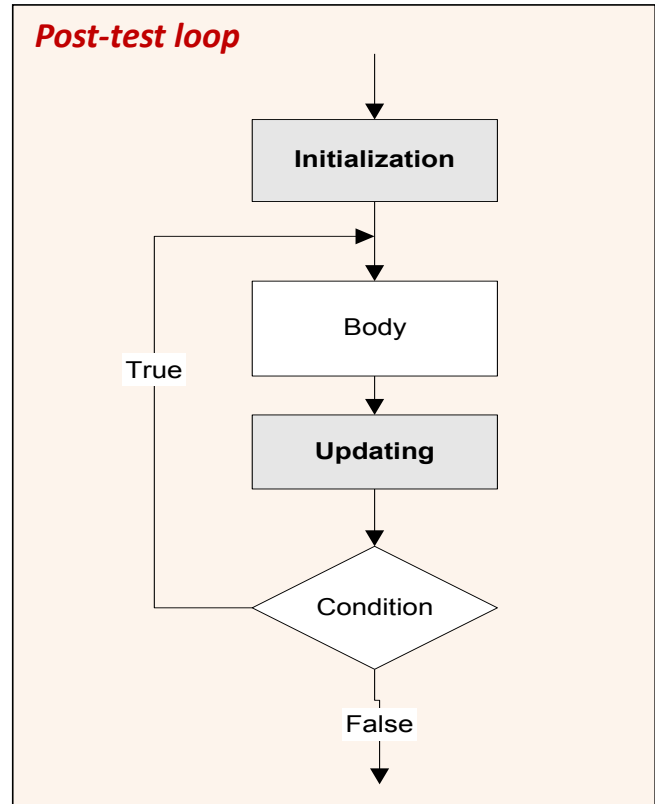
Parts of a Loop

- **Initialization**

- Is used to prepare a loop before it can start $\Rightarrow$ usually, here we **initialize the condition.**
- The initialization must be **written outside** of the loop, before the first execution of the body.

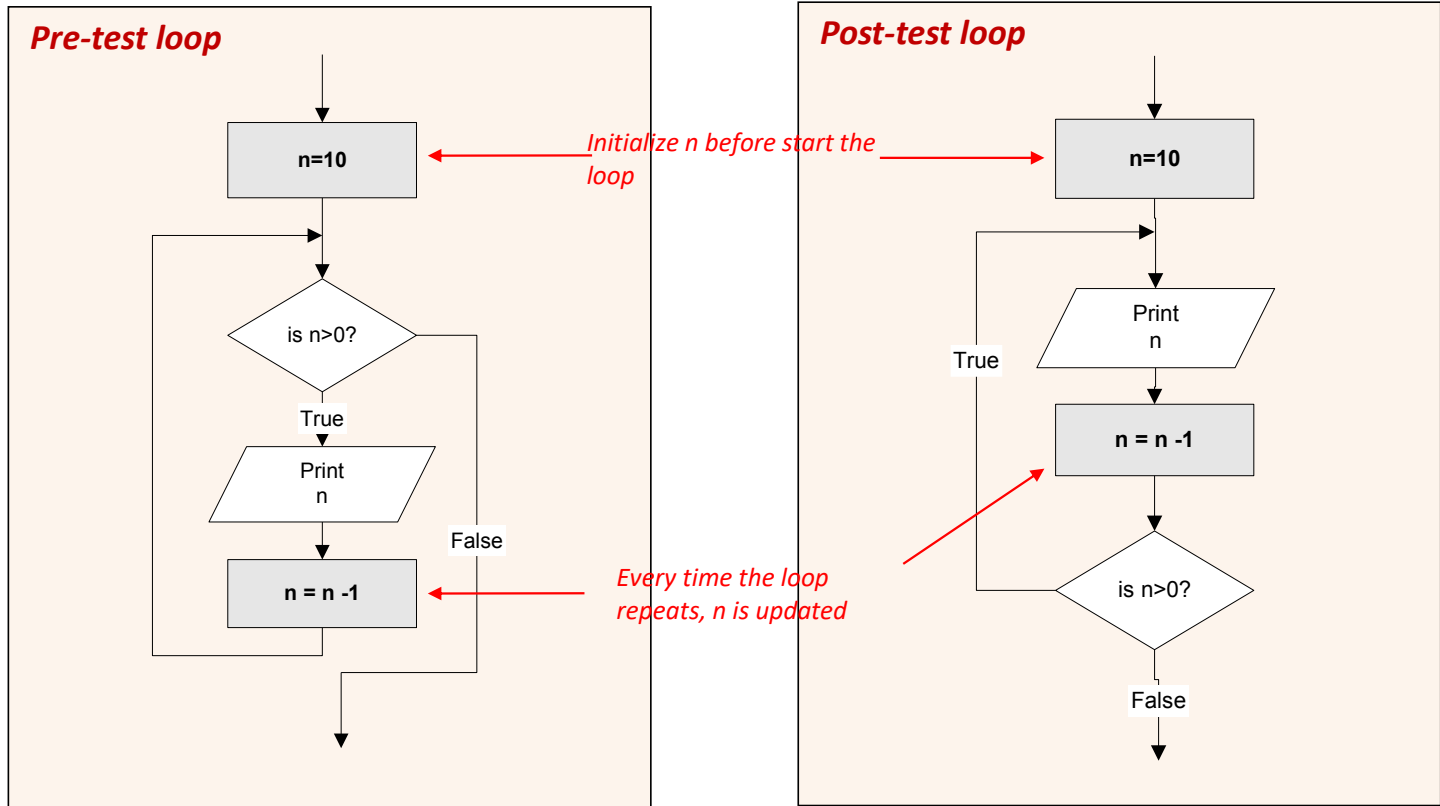
- **Updating**

- Is used **to update the condition.**
- If the condition is not updated, it always true $\Rightarrow$ the loop always repeats (an **infinite loop**)
- The updating part is **written inside** the loop. It is actually a **part of the body.**



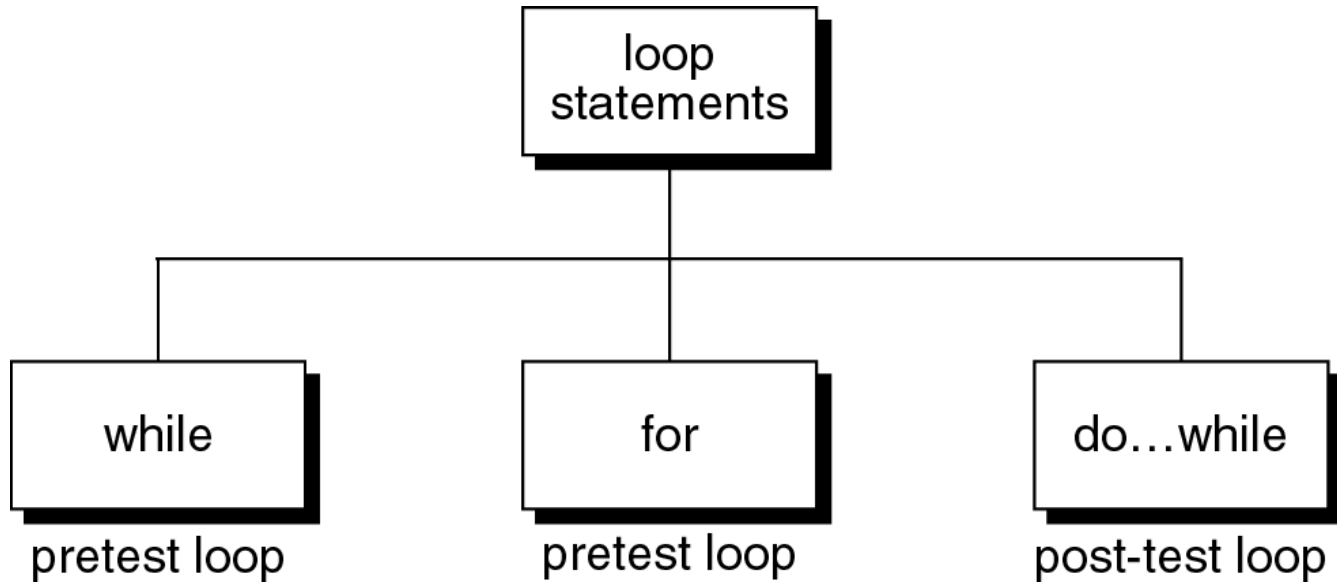
Example 1: Parts of a Loop

- These flowcharts print numbers 10 down to 1.



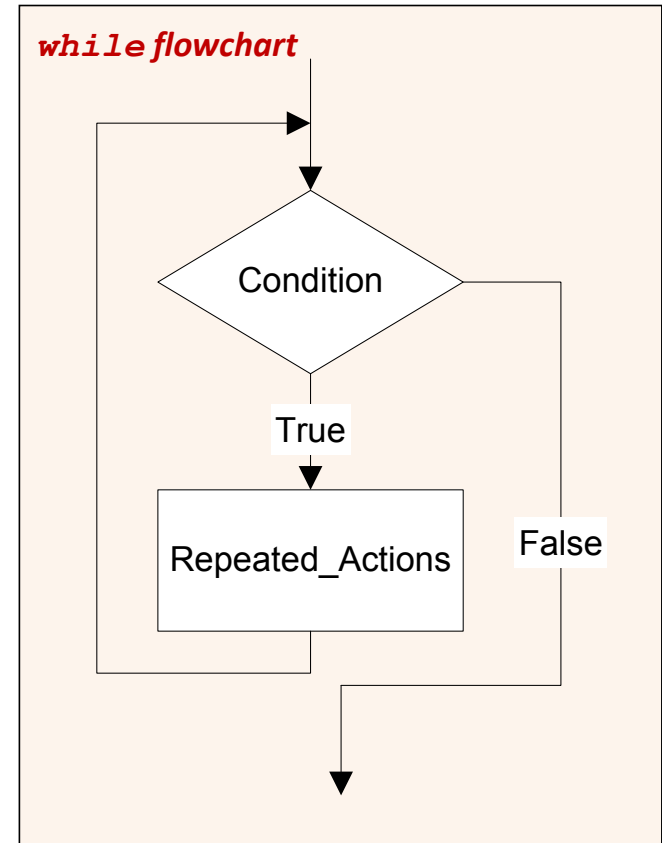
Loop Statements

- C++ provides three loop statements:



while Statement

```
while (condition)
{
    Repeated_actions;
}
```



while Statement

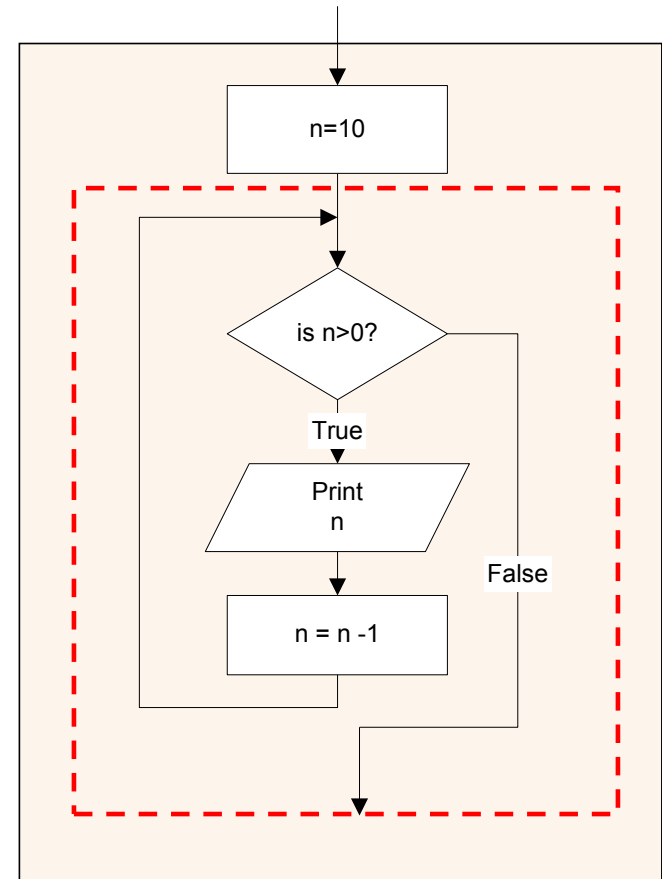
Example: Print numbers 10 down to 1 using **while** statement.

Note: The first line (**n = 10**) is actually not a part of the loop statement.

```
n = 10;
while (n > 0)
{
    cout << n << " ";
    n = n-1;
}
```

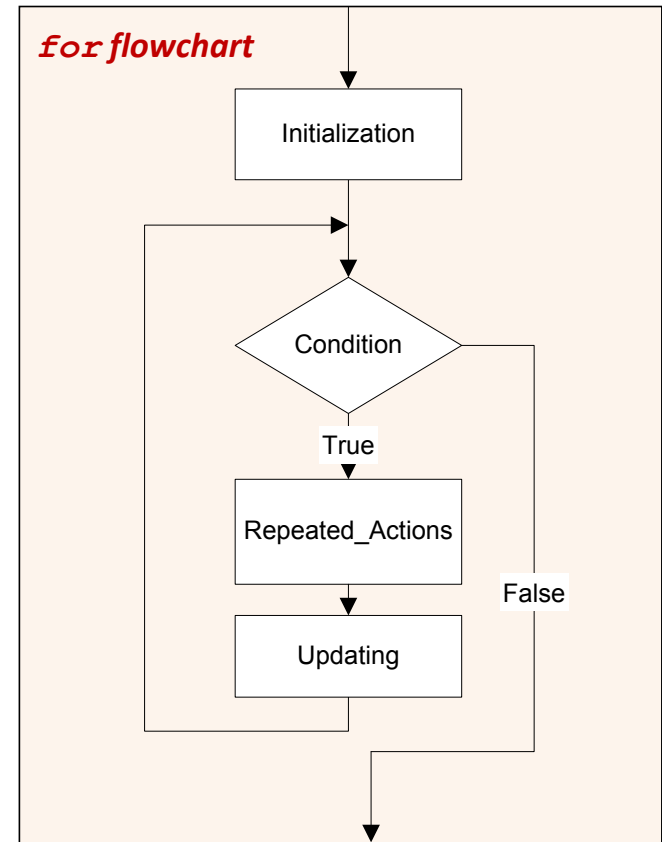
Output:

10 9 8 7 6 5 4 3 2 1



for Statement

```
for (Initialization;  
      Condition; Updating)  
{  
    Repeated_actions;  
}
```



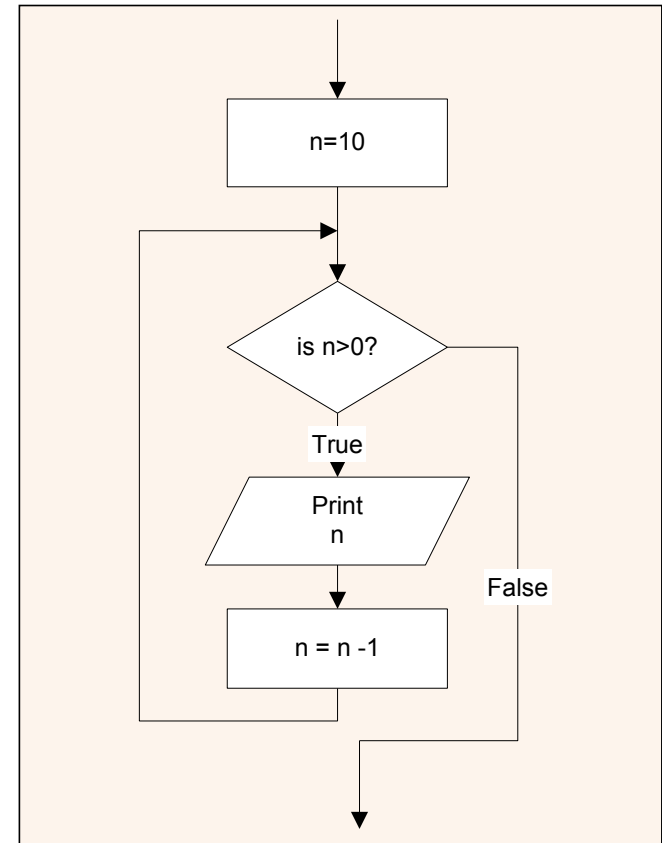
for Statement

Example: Print numbers 10 down to 1 using **for** statement.

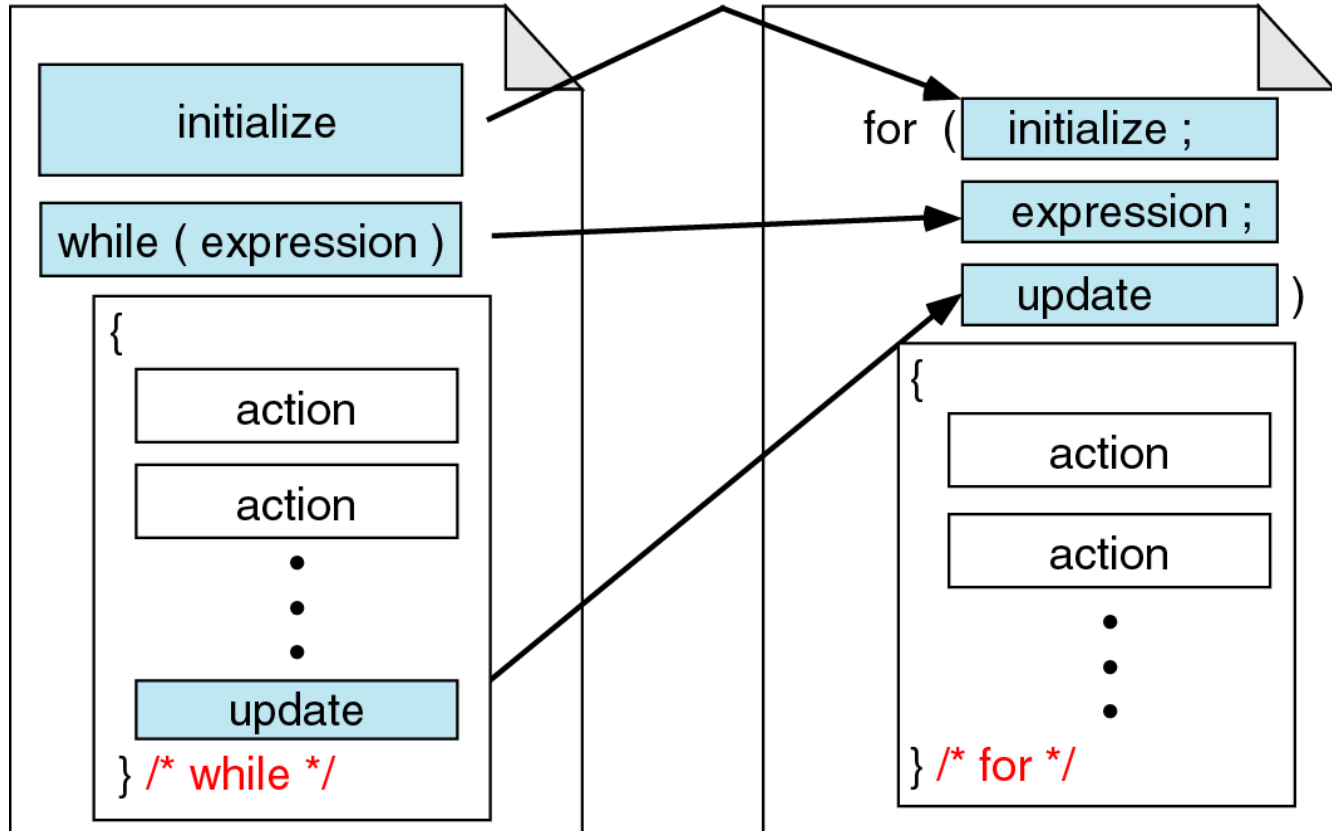
```
for(n = 10; n > 0; n = n - 1)
{
    cout << n << " ";
}
```

Output:

10 9 8 7 6 5 4 3 2 1



for vs. while Statements



Comparing for and while loops

do..while Statement

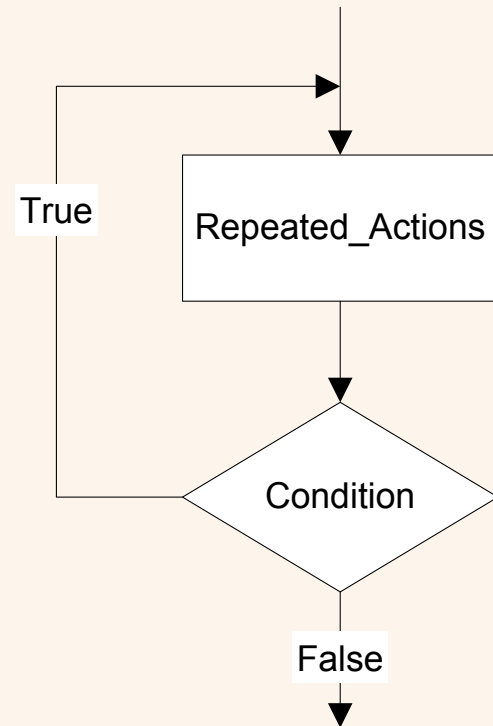
do

{

Repeated_actions;

} **while** (condition);

do..while flowchart



do..while Statement

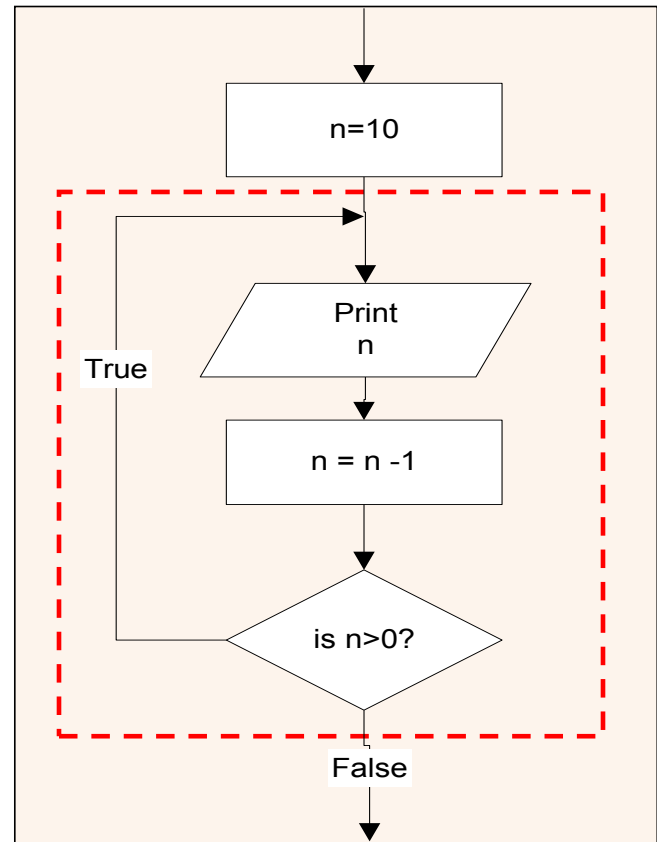
Example: Print numbers 10 down to 1 using **do..while** statement.

Note: The first line (**n = 10**) is actually not a part of the loop statement.

```
n = 10;  
do  
{  
    cout << n << " ";  
    n = n-1;  
} while (n > 0);
```

Output:

10 9 8 7 6 5 4 3 2 1



Loop Statements

- If the body part has only one statement, then the bracket symbols, **{ }** may be omitted.
- Example:

These two **for** statements are equivalent.

```
for(n = 10; n > 0; n = n - 1)
{
    cout << n;
}
```

```
for(n = 10; n > 0; n = n - 1)
    cout << n;
```

Breaking Out of a Loop

- Can use **break** to terminate execution of a loop.
- When used in an inner loop, terminates that loop only (inner loop) and returns to the outer loop.

```
while (condition)
{
    ...
    for ( ...; ...; ... )
```

```
{
    ...
    if (otherCondition)
        break;
    ...
} /* for */
```

```
/* more while processing */
```

```
} /* while */
```

The break statement takes you out of the inner loop (the *for* loop). The *while* loop is still active.

Exercise 1

- Determine the output of each code segment below:

```
✓ int x = 10;
  do {
    x++;
    if (x % 2 == 1 ) continue;
    cout << x << ", ";
  } while (x <= 21);
```

```
✓ int y = 2;
  do {
    if (y > 500 ) break;
    cout << y << ", ";
    y *= 3;
  } while (y > 1);
```

Exercise 2

- Rewrite the following program segment using a **do..while** loop:

```
int count = 0, x;
int n = 5;
int i = 0;
while (i < n)
{
    cin >> x;
    if (x == i)
        ++count;
    ++i;
}
cout << "count = " << count;
```

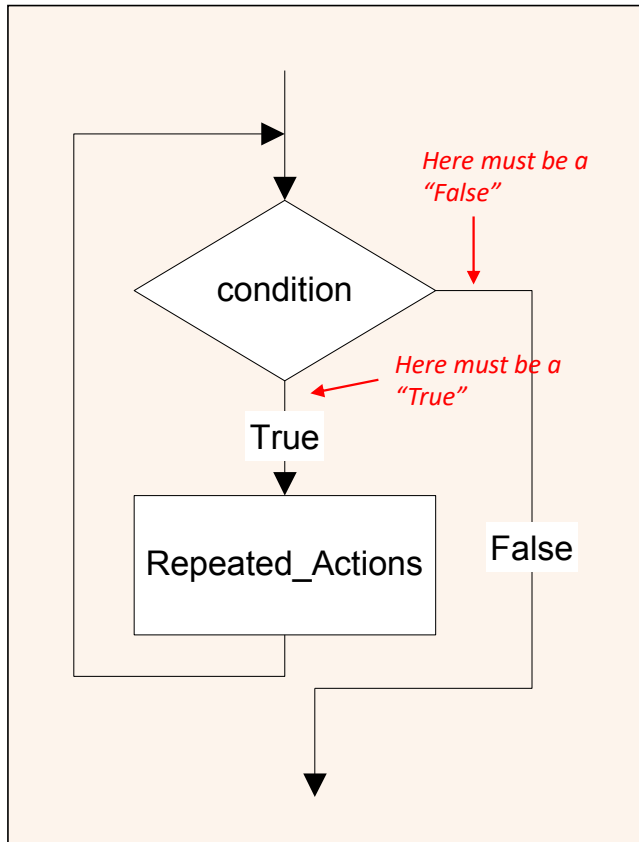
Exercise 3

- Rewrite the following code fragment so that it uses a **while** loop to accomplish the same task:

```
int n;  
do  
{  
    cout << "Enter a non-negative integer: ";  
    cin >> n;  
    if (n < 0)  
        cout << "The integer you entered is "  
            << "negative." << endl;  
} while (n < 0);
```

Translating Flowchart to C++ Code

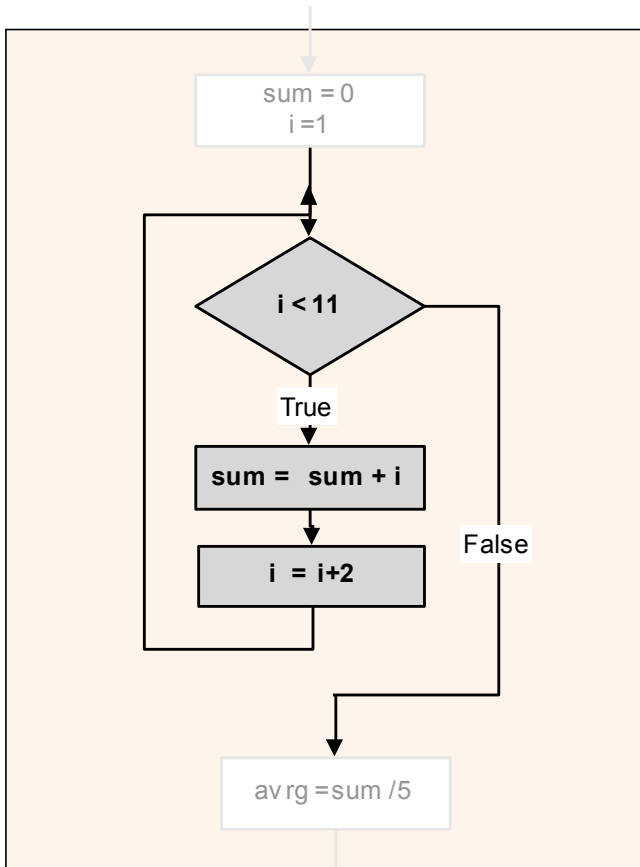
- Pattern 1



```
while (condition)
{
    Repeated_actions;
}
```

Translating Flowchart to C++ Code

- Example of pattern 1: Calculate the average of odd numbers 1 to 9.

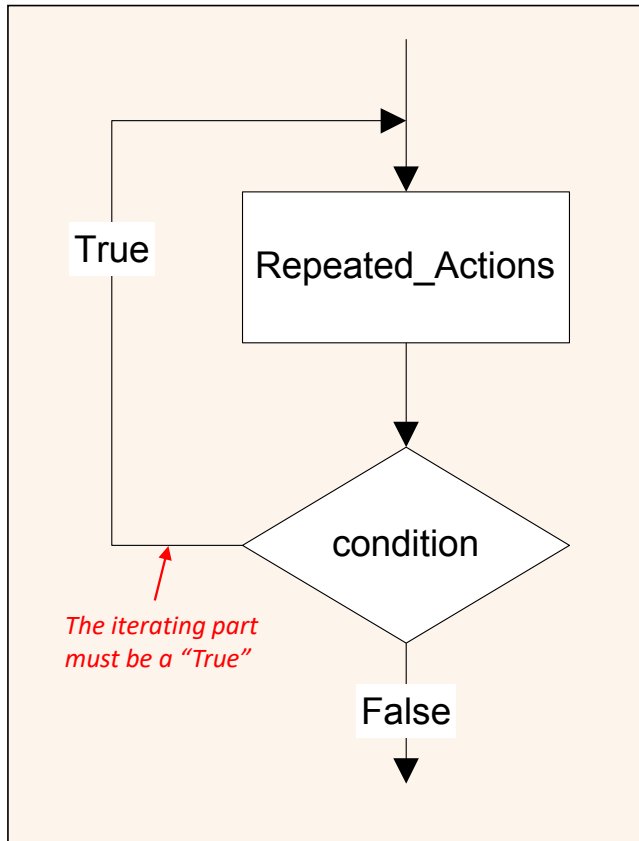


```

sum = 0;
i = 1;
while (i < 11)
{
    sum = sum + i;
    i = i + 2;
}
avrg = sum / 5.0;
  
```


Translating Flowchart to C++ Code

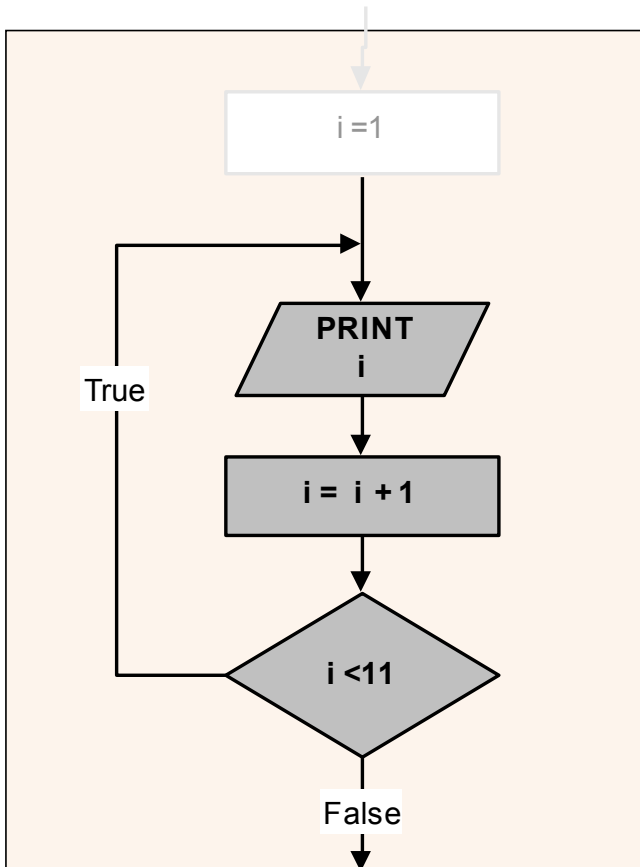
- Pattern 2



```
do
{
    Repeated_actions;
} while (condition);
```

Translating Flowchart to C++ Code

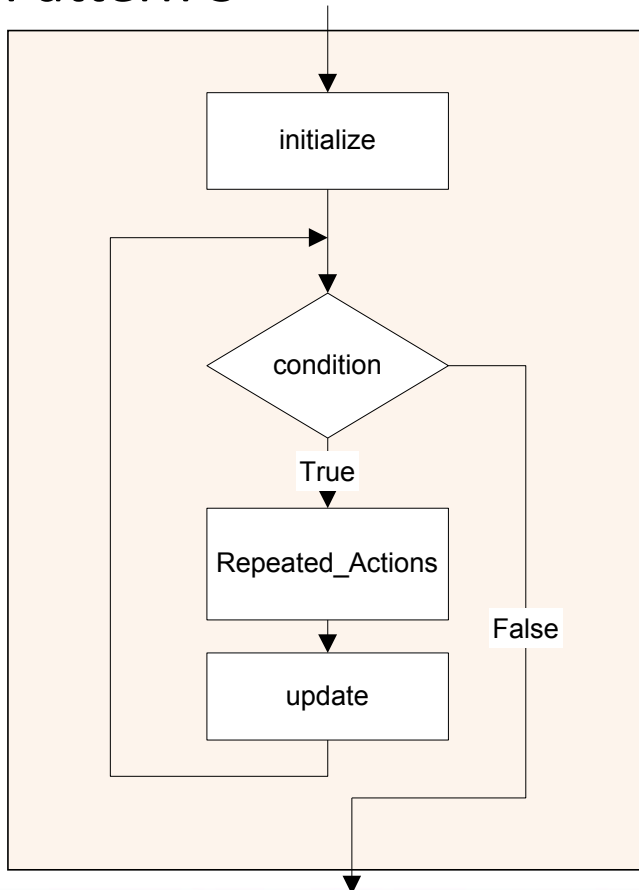
- Example of pattern 2: Prints numbers 1 to 10.



```
i = 1;  
do  
{  
    cout << i << endl;  
    i = i + 1;  
} while (i < 11);
```

Translating Flowchart to C++ Code

- Pattern 3

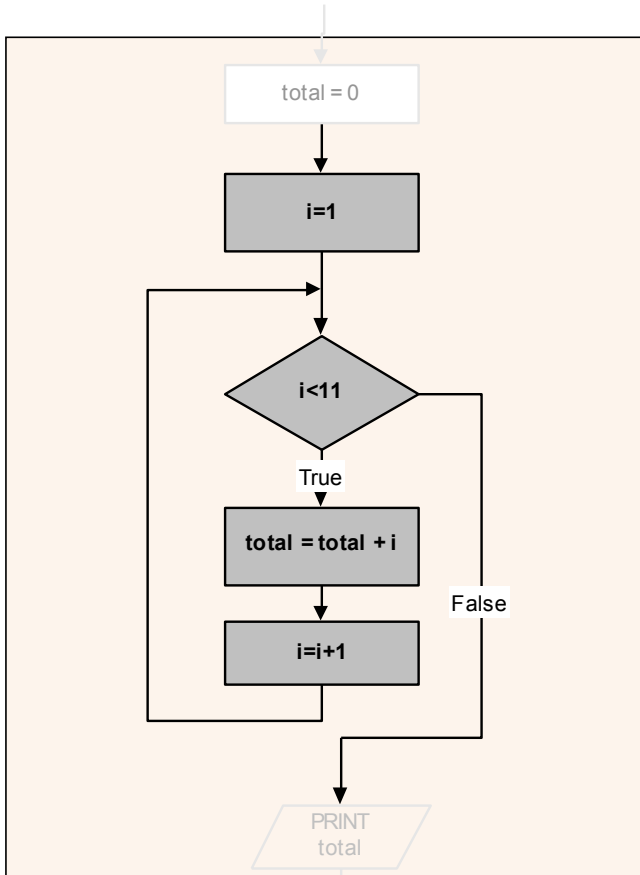


```
for(initialize;  
condition; update)  
{  
    Repeated_Actions;  
}
```

```
initialize;  
while(condition)  
{  
    Repeated_Actions;  
    update;  
}
```

Translating Flowchart to C++ Code

- Example of pattern 3: Print the total of numbers 1 to 10.



```

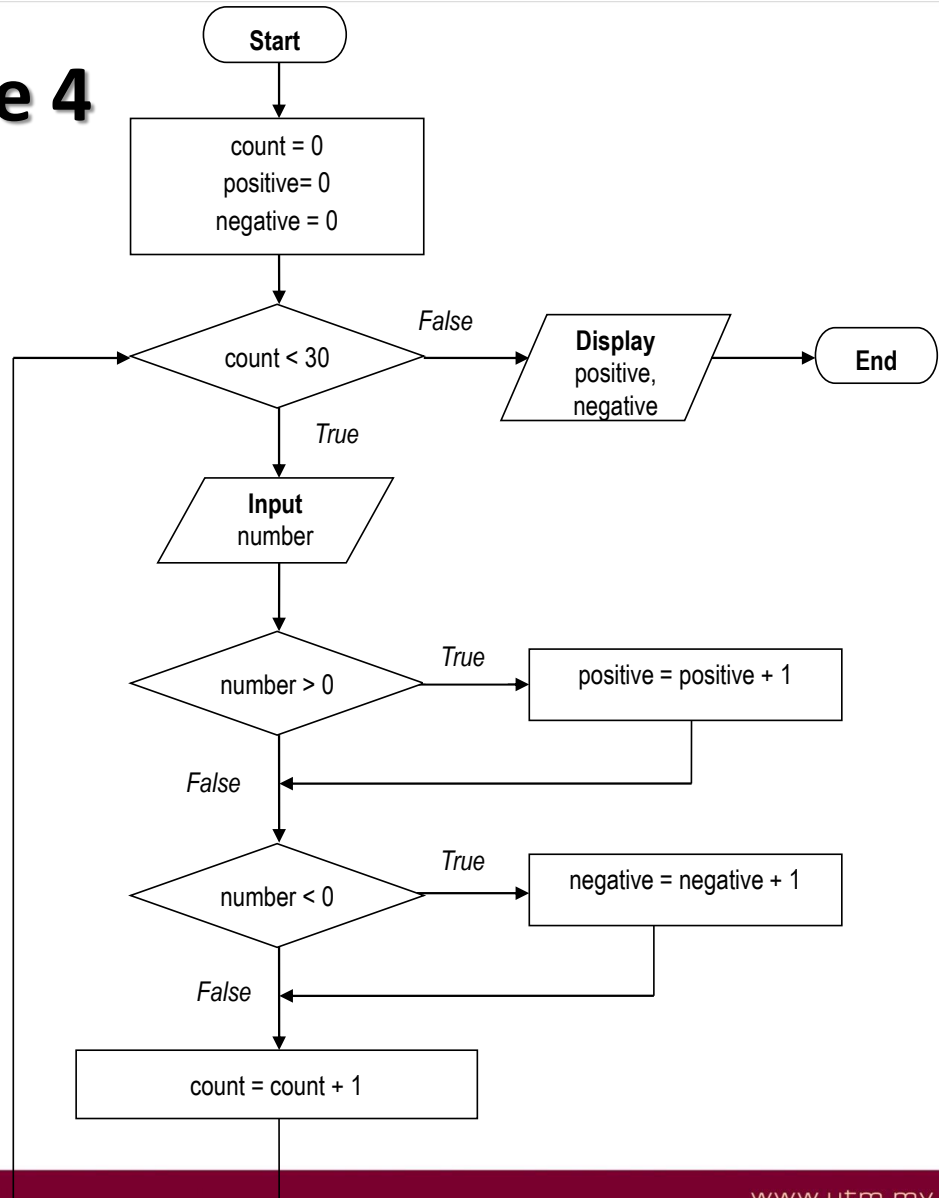
total = 0;
for (i = 1; i < 11; i++)
{
    total = total + i;
}
cout << total;
  
```

```

total = 0;
i = 1;
while (i < 11)
{
    total = total + i;
    i++;
}
cout << total;
  
```

Exercise 4

- The following flowchart presents an algorithm of a program that counts the number of positive and negative numbers entered by the user. The user will enter 30 numbers. Translate the flowchart into its corresponding C++ program using an appropriate loop statement.



Exercise 4 (cont.)

- Write your code in the incomplete program given below:

```
#include <iostream>
using namespace std;

int main()
{
    int count = 0;    //The total count of numbers entered
    int positive = 0; //The count of positive numbers
    int negative = 0; //The count of negative numbers
    int number;       //The number entered

    //Write your code here
    //...
    //...
    //...
    return 0;
}
```

Deciding Which Loop to Use

- **while**: pretest loop (loop body may not be executed at all).
- **do...while**: post test loop (loop body will always be executed at least once).
- **for**: pretest loop (loop body may not be executed at all); has initialization and update code; is useful with counters or if precise number of repetitions is known

Nested Loops

- A **nested loop** is a loop inside the body of another loop.
- Example:

```
for (row = 1; row <= 3; row++)  
{  
    for (col = 1; col <= 3; col++)  
    {  
        cout << row * col << endl;  
    }  
}
```

outer loop

inner loop

Notes on Nested Loops

- Inner loop goes through all its repetitions for each repetition of outer loop.
- Inner loop repetitions complete sooner than outer loop.
- Total number of repetitions for inner loop is product of number of repetitions of the two loops. In previous example, inner loop repeats 9 times.

Exercise 5

- How many times the outer loop is executed? How many times the inner loop is executed? What is the output?

```
#include <iostream>
using namespace std;
int main()
{
    int x, y;
    for(x = 1; x <= 8; x += 2)
        for(y = x; y <= 10; y += 3)
            cout << "\nx = " << x
                << "    y = " << y;
    return 0;
}
```

Exercise 6

- Determine the output of each code segment below:

```
for (int i = 1; i < 4; i++)  
{  
    int j = i;  
    while (j > 0)  
    {  
        cout << "i = " << i  
              << ", j = " << j << endl;  
        j--;  
    }  
}
```